

Project 02 — Online Computer Shop

Course: Web Technologies | **Format:** Project | **Duration:** 1 week

Application Overview

An “Online Computer Shop” where different PC peripherals and components are shown with detailed manufacturing information and price. Users can browse, view components, add products to cart, and select payment methods. Two user roles: **ADMIN** and **CUSTOMER**.

- **ADMIN:** main controlling point – removes customers and their reviews.
- **CUSTOMER:** browses products, searches with filters, posts reviews, adds to cart, selects payment method, and places orders.
- **Unregistered visitors** can browse but cannot post reviews or order.

Team Structure & Student IDs

Student ID	Task	Main Features
22-48356-3	Task 1	User Authentication, Registration, Profile, Home Page, Category Bar, Featured Components
22-48412-3	Task 2	Admin – Category & Sub-category Management, Brand Management, Product Management (full CRUD)
22-48484-3	Task 3	Customer – Browse by Category/Sub-category/Brand, Search & Filtering (AJAX), Product Details, Cart (add/view/update/remove)
22-49926-3	Task 4	Customer Reviews (post/delete own), Order Placement with Payment Method Selection, Admin Removal of Customers & Reviews

Shared Database Schema (same for all)

Table	Key Columns
users	id, name, email, password_hash, role (admin/customer), profile_picture, created_at
categories	id, name, parent_id (for sub-categories), created_at
brands	id, name, category_id, created_at
products	id, name, description, manufacturer_review, price, category_id, brand_id, image_path, stock, created_at
cart	id, user_id, product_id, quantity, added_at
orders	id, user_id, total_amount, payment_method (cash_on_delivery/online_wallet), status, order_date
order_items	id, order_id, product_id, quantity, unit_price
reviews	id, product_id, user_id, reviewer_name, comment, created_at

No code sharing except the schema.

Global Technical Requirements

- PHP MVC (controllers/, models/, views/, config/)
- Passwords hashed with `password_hash()` / verified with `password_verify()`
- PDO/mysqli + **prepared statements** for every query – no string-concatenated SQL
- Server-side validation on every form before any DB write; show inline error messages
- `session_start()` on every page that requires auth; redirect unauthenticated users
- AJAX endpoints return Content-Type: application/json; handle errors client-side
- File uploads go to `public/uploads/` (profile pictures, product images) – validate MIME type + size server-side
- Do not drop or alter the shared schema tables

Mandatory Grading Criteria (each student must satisfy all 10 within their task)

1. **Basic Web Security** – SQL injection prevention, XSS protection, CSRF awareness, secure password storage.
2. **UI (HTML/CSS)** – Clean, responsive, user-friendly interface.
3. **Feature Completeness** – All assigned requirements work error-free.
4. **DB** – Correct use of shared schema, proper relationships, data integrity.
5. **Auth (Session/Cookie)** – Session management, role-based access, “Remember Me” (optional).
6. **MVC** – Clear separation of business logic, presentation, and request handling.
7. **JS Validation** – Client-side validation on forms (e.g., registration, product search, review, cart).
8. **PHP Validation** – Server-side validation on every input before DB write.
9. **Ajax/JSON** – At least one AJAX endpoint returning JSON per student.
10. **Git Contribution** – Feature branches, meaningful commits (≥ 3 per student), merge into `main` via PR.

Git Flow

- Repository has a `main` branch (protected, no direct pushes).
- Each student creates a **feature branch** from `main` named `feature/taskX-studentID` (e.g., `feature/task1-2249686-3`).
- Work on the feature branch, commit regularly (at least 3 commits per student).
- After completing the feature, open a **pull request** into `main` and merge.
- Final `main` must contain all four feature branches merged with full history.

Task Breakdown

Task 1 – User Authentication, Registration, Profile, Home Page, Category Bar & Featured Components

Student: 22-48356-3

CRUD included: Create (registration), Read/Update (profile).

Requirements:

- **Registration** – form for both admin and customers. Collect: name, email, password (≥ 8 chars), role (admin/customer). Validate unique email, hash password. Redirect to login with flash message.

- **Login & Remember Me** – login creates `$_SESSION['user_id']`, `$_SESSION['name']`, `$_SESSION['role']`. “Remember Me” checkbox stores secure random token (hashed) in `users.remember_token` and writes 30-day cookie. On subsequent visits PHP reinstates session automatically.
- **Profile Page** (session-gated) – user can update name, email, profile picture (upload to `public/uploads/`), and change password (require current password). Show success banner on save.
- **Logout & Navbar** – logout destroys session and deletes remember cookie. Navbar shows different links based on role (admin/customer/guest).
- **Home Page** – for all visitors:
 - Category bar (top navigation – list all top-level categories from DB). Clicking a category navigates to that category page.
 - Featured components section – fetch 4–6 random or latest products, display name, small manufacturer review, and price.
- **JS validation** on registration and profile forms (e.g., password match, email format).

Key outputs: `users` table populated with admin and customer accounts; session + “Remember Me” functional; home page displays dynamic categories and featured products.

Task 2 – Admin: Category & Sub-category Management, Brand Management, Product Management (full CRUD)

Student: 22-48412-3

CRUD included: Full CRUD on categories (including sub-categories), brands, and products.

Requirements:

- **Admin Gate** – every admin page checks `$_SESSION['role'] === 'admin'`; redirect others.
- **Category Management** – create, edit, delete categories. Support sub-categories (`parent_id` field). Example: Storage → Permanent storage (HDD, SSD) and Portable storage. Deleting a category blocks if it has child categories or products (show descriptive error).
- **Brand Management** – create, edit, delete brands under a specific category. Example: Monitor category → brands ASUS, LG, DELL. Deleting a brand blocks if products exist under that brand.
- **Product Management** – create, edit, delete products. Form includes: name, description, manufacturer review (text), price (>0), category dropdown, brand dropdown (populated based on selected category), image upload (JPEG/PNG $\leq 2\text{MB}$ to `public/uploads/products/`), stock quantity. Edit pre-fills form. Delete removes image file.
- **Dashboard summary** – total products, total categories, total brands, low-stock alerts (e.g., `stock < 5`).
- **PHP & JS validation** on all forms (e.g., price positive, image type/size).

- **AJAX** – optional for dynamic brand loading when category changes, or inline product status toggle (e.g., active/inactive).

Key outputs: Fully populated categories, brands, products tables; admin can manage entire inventory.

Task 3 – Customer: Browse by Category/Sub-category/Brand, Search & Filtering (AJAX), Product Details, Cart

Student: 22-48484-3

CRUD included: Create, Update, Delete on cart (session or DB based cart). Read for product browsing.

Requirements:

- **Customer Gate** – cart and order actions require login; browsing is public.
- **Browse by Category/Sub-category/Brand** – when user clicks a category (e.g., Ram), show all products from that category and all its sub-categories and all brands under them. Similarly, clicking a sub-category or brand shows only relevant products. Use clean URLs like /category/ram, /brand/dell.
- **Component Pages** – each page shows product name, small manufacturer review, price. Pagination optional.
- **Search Box & Filtering (AJAX)** – search box on every page. On submit or keystroke, call GET /api/products/search?q=.... Filtering options: price range (min/max), category, brand. AJAX returns JSON, JS dynamically updates product grid without page reload.
- **Product Detail Page** – displays full description, manufacturer review, price, stock status, image. “Add to Cart” button.
- **Cart Management (AJAX)** –
 - Add to cart (AJAX POST /api/cart/add) – updates session or DB cart, returns new cart item count.
 - Cart page – list items (product name, quantity, unit price, subtotal).
 - Update quantity (+/-) and remove item via AJAX calls (/api/cart/update, /api/cart/remove). Returns updated totals.
 - Show total price in cart.
- **JS validation** on search filters (price range as numbers) and cart quantity (positive integer).
- **PHP validation** – ensure product exists, quantity not exceeding stock.

Key outputs: Customers can browse by category/sub-category/brand, search/filter products, view details, and manage cart entirely with AJAX.

Task 4 – Customer Reviews, Order Placement with Payment Method, Admin Removal of Customers & Reviews

Student: 22-49926-3

CRUD included: Create/Delete (reviews by customer), Create (order), Delete (users and reviews by admin).

Requirements:

- **Review Section** – under every product detail page.
 - Display existing reviews (reviewer name, comment, date).
 - Only logged-in customers can post a review. Form pre-fills name from profile, comment field.
 - Submit review (AJAX POST /api/reviews/add) – saves to reviews with user_id.
 - Customer can delete their own review (AJAX DELETE /api/reviews/{id}).
- **Order Placement** – from cart page, customer selects payment method (cash on delivery / online wallet). Checkout process:
 - Validate cart not empty.
 - Create orders row (user_id, total_amount, payment_method, status='pending', order_date).
 - Create order_items rows for each cart item.
 - Clear cart.
 - Redirect to order confirmation page showing order ID and summary.
- **Admin – Remove Customers** – admin page listing all customers (role='customer'). Each row has “Delete” button. Delete cascades: remove customer’s reviews, cart items, orders (or set user_id to NULL with caution). Use AJAX or POST confirm.
- **Admin – Remove Reviews** – admin page listing all reviews (with product name, reviewer name). Delete button removes any review (AJAX).
- **Admin Dashboard** – additional section showing recent orders and recent reviews.
- **JS validation** on review form (non-empty comment, max length).
- **PHP validation** – ensure only logged-in customers can post reviews; admin can delete any user/review.

Key outputs: Full review system (post/delete own), order placement with payment method selection, admin ability to remove any customer or review. All CRUD covered.